

Declarações Sequenciais

Circuitos Combinacionais

Prof. Roberto de Matos

roberto.matos@ifsc.edu.br



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
São José

- Entender os conceitos de declarações sequenciais em VHDL:
 - *Process* em VHDL
 - Atribuição de sinal sequencial
 - Atribuição de variável
 - *IF*, *CASE* e *LOOP*
 - Diretrizes de Implementação

Leitura Recomendada:

Capítulo 5 do livro: *RTL Hardware Design Using VHDL*



Process

- Contém um conjunto de instruções a serem executadas sequencialmente.
- Todo o *process* é considerado uma única instrução concorrente.
- Pode ser interpretado como uma caixa preta.
- Pode ou não ser capaz de ser mapeado para *hardware* físico.

Lembrete:

Instruções sequenciais $\neq$ Circuitos sequenciais.



■ Sintaxe utilizando lista de sensibilidade:

```
1    process(lista de sensibilidade)
2        declarações;
3    begin
4        instruções sequenciais;
5        instruções sequenciais;
6        . . .
7    end process;
8
```



- Problema: Lista de sensibilidade incompleta.

```
1    signal a,b,c,y: std_logic;  
2    ...  
3    process (a)  
4    begin  
5        y <= a and b and c;  
6    end process;  
7
```



- Problema: Lista de sensibilidade incompleta.

```
1    signal a,b,c,y: std_logic;  
2    ...  
3    process (a)  
4    begin  
5        y <= a and b and c;  
6    end process;  
7
```

Lembrete:

Para circuitos combinacionais, **todas** as entradas devem estar na lista de sensibilidade.



Declaração de atribuição

Declaração de atribuição de SINAL

- Sintaxe:

```
1 signal_name <= value_expression;
```

- Não é possível criar uma forma de onda (*projected-waveform*) igual a declaração concorrente.
- Várias atribuições podem ser feitas, mas somente a última atribuição tem efeito.



Declaração de atribuição de SINAL

■ Sintaxe:

```
1 signal_name <= value_expression;
```

- Não é possível criar uma forma de onda (*projected-waveform*) igual a declaração concorrente.
- Várias atribuições podem ser feitas, mas somente a última atribuição tem efeito.
- Exemplo:

```
1 process(a,b,c,d)  
2 begin  
3     y <= a or c;  
4     y <= a and b;  
5     y <= c and d;  
6 end process;
```



Declaração de atribuição de SINAL

■ Sintaxe:

```
1 signal_name <= value_expression;
```

- Não é possível criar uma forma de onda (*projected-waveform*) igual a declaração concorrente.
- Várias atribuições podem ser feitas, mas somente a última atribuição tem efeito.
- Exemplo:

```
1 process(a,b,c,d)
2 begin
3     y <= a or c;
4     y <= a and b;
5     y <= c and d;
6 end process;
```

```
1 process(a,b,c,d)
2 begin
3     y <= c and d;
4 end process;
```



- O que ocorre se as três instruções estivessem fora do *process*?

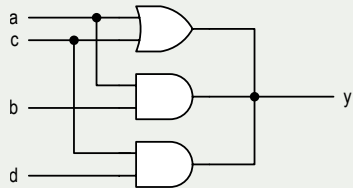
```
1   y <= a or c;  
2   y <= a and b;  
3   y <= c and d;  
4
```



Declaração de atribuição de SINAL

- O que ocorre se as três instruções estivessem fora do *process*?

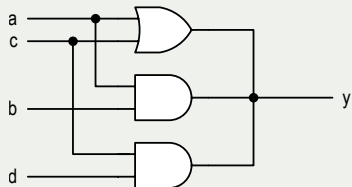
```
1  y <= a or c;  
2  y <= a and b;  
3  y <= c and d;  
4
```



Declaração de atribuição de SINAL

- O que ocorre se as três instruções estivessem fora do *process*?

```
1  y <= a or c;  
2  y <= a and b;  
3  y <= c and d;  
4
```



Lembrete:

Atribuição condicional (*when else*) e de seleção (*with select else*) **NÃO** podem ser utilizadas dentro do *process*.



Declaração de atribuição de **VARIÁVEL**

- Sintaxe:

```
1 variable_name := value_expression;
```

- A atribuição entra em vigor imediatamente.
- Sem dimensão de tempo (ou seja, sem atraso).
- Comporta-se como variáveis em C.
- Difícil de mapear para hardware.



Declaração de atribuição de **VARIÁVEL**

- Sintaxe:

```
1 variable_name := value_expression;
```

- A atribuição entra em vigor imediatamente.
- Sem dimensão de tempo (ou seja, sem atraso).
- Comporta-se como variáveis em C.
- Difícil de mapear para hardware.

Dica:

NÃO USE!!



Declaração *IF*

■ Sintaxe:

```
1      if boolean_expr_1 then
2          sequential_statements;
3      elsif boolean_expr_2 then
4          sequential_statements;
5      elsif boolean_expr_3 then
6          sequential_statements;
7      . . .
8      else
9          sequential_statements;
10     end if;
11
```



■ Sintaxe:

```
1      if boolean_expr_1 then
2          sequential_statements;
3      elsif boolean_expr_2 then
4          sequential_statements;
5      elsif boolean_expr_3 then
6          sequential_statements;
7      . . .
8      else
9          sequential_statements;
10     end if;
11
```

■ Exemplo (Mux 4x1):

```
1      architecture if_arch of mux4 is
2      begin
3          process(a,b,c,d,s)
4          begin
5              if (s="00") then
6                  x <= a;
7              elsif (s="01") then
8                  x <= b;
9              elsif (s="10") then
10                 x <= c;
11             else
12                 x <= d;
13             end if;
14         end process;
15     end if_arch;
16
```



- As duas estruturas são iguais se houver apenas um sinal de saída no *IF*.
- O *IF* é mais flexível.
- As instruções sequenciais podem ser usadas nos ramos *then*, *elsif* e *else*:
 - Múltiplas instruções;
 - *IF* aninhados.



IF vs. atribuição CONDICIONAL (*when else*)

- A seguinte declaração de atribuição CONDICIONAL:

```
1 sig <= value_expr_1 when boolean_expr_1 else
2     value_expr_2 when boolean_expr_2 else
3     value_expr_3 when boolean_expr_3 else
4     ...
5     value_expr_n;
```

- Pode ser escrita como:

```
1 if boolean_expr_1 then
2     sig <= value_expr_1;
3 elsif boolean_expr_2 then
4     sig <= value_expr_2;
5 elsif boolean_expr_3 then
6     sig <= value_expr_3;
7 . . .
8 else
9     sig <= value_expr_n;
10 end if;
```



IF vs. atribuição CONDICIONAL (*when else*)

- Suponha que queremos encontrar o maior valor dentre três sinais.

- IF:

```
1 process(a,b,c)
2 begin
3   if (a > b) then
4     if (a > c) then
5       max <= a;    -- a>b and a>c
6     else
7       max <= c;    -- a>b and c>=a
8     end if;
9   else
10    if (b > c) then
11      max <= b;    -- b>=a and b>c
12    else
13      max <= c;    -- b>=a and c>=b
14    end if;
15  end if;
16 end process;
```



IF vs. atribuição CONDICIONAL (*when else*)

- Suponha que queremos encontrar o maior valor dentre três sinais.

- *IF*:

```
1 process(a,b,c)
2 begin
3   if (a > b) then
4     if (a > c) then
5       max <= a;  -- a>b and a>c
6     else
7       max <= c;  -- a>b and c>=a
8     end if;
9   else
10    if (b > c) then
11      max <= b;  -- b>=a and b>c
12    else
13      max <= c;  -- b>=a and c>=b
14    end if;
15  end if;
16 end process;
```

- *when else* (3 sinais) :

```
1 signal ac_max , bc_max : std_logic ;
2 ...
3 ac_max <= a when (a > c) else c;
4 bc_max <= b when (b > c) else c;
5 max <= ac_max when (a > b) else bc_max;
```



IF vs. atribuição CONDICIONAL (*when else*)

- Suponha que queremos encontrar o maior valor dentre três sinais.

- *IF*:

```
1 process(a,b,c)
2 begin
3   if (a > b) then
4     if (a > c) then
5       max <= a;  -- a>b and a>c
6     else
7       max <= c;  -- a>b and c>=a
8     end if;
9   else
10    if (b > c) then
11      max <= b;  -- b>=a and b>c
12    else
13      max <= c;  -- b>=a and c>=b
14    end if;
15  end if;
16 end process;
```

- *when else* (3 sinais) :

```
1 signal ac_max , bc_max : std_logic ;
2 ...
3 ac_max <= a when (a > c) else c;
4 bc_max <= b when (b > c) else c;
5 max <= ac_max when (a > b) else bc_max;
```

- *when else* (“nivelando”):

```
1 max <= a when ((a > b) and (a > c)) else
2       c when (a > b) else
3       b when (b > c) else c;
```



IF vs. atribuição CONDICIONAL (*when else*)

- Outra situação adequada para a instrução *IF* é quando muitas operações são controladas pela mesma condição booleana.

- *IF*:

```
1 process(a,b)
2 begin
3   if (a > b and op="00") then
4     y <= a-b;
5     z <= a-1;
6     status <= '0';
7   else
8     y <= b-a;
9     z <= b-1;
10    status <= '1';
11  end if;
12 end process;
```



IF vs. atribuição CONDICIONAL (*when else*)

- Outra situação adequada para a instrução *IF* é quando muitas operações são controladas pela mesma condição booleana.

■ *IF*:

```
1 process(a,b)
2 begin
3   if (a > b and op="00") then
4     y <= a-b;
5     z <= a-1;
6     status <= '0';
7   else
8     y <= b-a;
9     z <= b-1;
10    status <= '1';
11  end if;
12 end process;
```

■ *when else* :

```
1 y <= a-b when (a > b and op="00") else
2   b-a;
3
4 z <= a-1 when (a > b and op="00") else
5   b-1;
6
7 status <= '0' when (a > b and op="00") else
8   '1';
```



IF ou atribuição de sinal incompletos

- De acordo com a definição VHDL:
 - Apenas o *then* é obrigatório; *elsif* e *else* são opcionais;
 - Os sinais não precisam ser atribuídos em todos os ramos;
 - Quando um sinal não é atribuído por omissão, ele mantém o “valor anterior” (*latch*).



IF ou atribuição de sinal incompletos

- De acordo com a definição VHDL:
 - Apenas o *then* é obrigatório; *elsif* e *else* são opcionais;
 - Os sinais não precisam ser atribuídos em todos os ramos;
 - Quando um sinal não é atribuído por omissão, ele mantém o “valor anterior” (*latch*).
- Exemplo:

```
1  process(a,b)
2  begin
3      if (a = b) then
4          eq <= '1';
5      end if;
6  end process;
```



IF ou atribuição de sinal incompletos

- De acordo com a definição VHDL:
 - Apenas o *then* é obrigatório; *elsif* e *else* são opcionais;
 - Os sinais não precisam ser atribuídos em todos os ramos;
 - Quando um sinal não é atribuído por omissão, ele mantém o “valor anterior” (*latch*).
- Exemplo:
- Significado:

```
1  process(a,b)
2  begin
3      if (a = b) then
4          eq <= '1';
5      end if;
6  end process;
```

```
1  process(a,b)
2  begin
3      if (a = b) then
4          eq <= '1';
5      else
6          eq <= eq;
7      end if;
8  end process;
```



IF ou atribuição de sinal incompletos

- De acordo com a definição VHDL:
 - Apenas o *then* é obrigatório; *elsif* e *else* são opcionais;
 - Os sinais não precisam ser atribuídos em todos os ramos;
 - Quando um sinal não é atribuído por omissão, ele mantém o “valor anterior” (*latch*).
- Exemplo:
- Significado:
- Correção:

```
1 process(a,b)
2 begin
3     if (a = b) then
4         eq <= '1';
5     end if;
6 end process;
```

```
1 process(a,b)
2 begin
3     if (a = b) then
4         eq <= '1';
5     else
6         eq <= eq;
7     end if;
8 end process;
```

```
1 process(a,b)
2 begin
3     if (a = b) then
4         eq <= '1';
5     else
6         eq <= '0';
7     end if;
8 end process;
```



IF ou atribuição de sinal incompletos

- De acordo com a definição VHDL:
 - Apenas o *then* é obrigatório; *elsif* e *else* são opcionais;
 - Os sinais não precisam ser atribuídos em todos os ramos;
 - Quando um sinal não é atribuído por omissão, ele mantém o “valor anterior” (*latch*).
- Exemplo:
- Significado:
- Correção:

```
1 process(a,b)
2 begin
3     if (a = b) then
4         eq <= '1';
5     end if;
6 end process;
```

```
1 process(a,b)
2 begin
3     if (a = b) then
4         eq <= '1';
5     else
6         eq <= eq;
7     end if;
8 end process;
```

```
1 process(a,b)
2 begin
3     if (a = b) then
4         eq <= '1';
5     else
6         eq <= '0';
7     end if;
8 end process;
```

Para um circuito combinacional, o *else* sempre deve ser incluído.



■ Exemplo:

```
1  process (a,b)
2  begin
3      if (a>b) then
4          gt <= '1';
5      elsif (a=b) then
6          eq <= '1';
7      else
8          lt <= '1';
9      end if;
10 end process;
```



IF ou atribuição de sinal incompletos

■ Exemplo:

```
1  process (a,b)
2  begin
3      if (a>b) then
4          gt <= '1';
5      elsif (a=b) then
6          eq <= '1';
7      else
8          lt <= '1';
9      end if;
10 end process;
```

■ Correção 1:

```
1  process (a,b)
2  begin
3      if (a>b) then
4          gt <= '1';
5          eq <= '0';
6          lt <= '0';
7      elsif (a=b) then
8          gt <= '0';
9          eq <= '1';
10         lt <= '0';
11     else
12         gt <= '0';
13         eq <= '0';
14         lt <= '1';
15     end if;
16 end process;
```



IF ou atribuição de sinal incompletos

■ Exemplo:

```
1  process (a,b)
2  begin
3      if (a>b) then
4          gt <= '1';
5      elsif (a=b) then
6          eq <= '1';
7      else
8          lt <= '1';
9      end if;
10 end process;
```

■ Correção 1:

```
1  process (a,b)
2  begin
3      if (a>b) then
4          gt <= '1';
5          eq <= '0';
6          lt <= '0';
7      elsif (a=b) then
8          gt <= '0';
9          eq <= '1';
10         lt <= '0';
11     else
12         gt <= '0';
13         eq <= '0';
14         lt <= '1';
15     end if;
16 end process;
```

■ Correção 2:

```
1  process (a,b)
2  begin
3      gt <= '0';
4      eq <= '0';
5      lt <= '0';
6      if (a>b) then
7          gt <= '1';
8      elsif (a=b) then
9          eq <= '1';
10     else
11         lt <= '1';
12     end if;
13 end process;
```



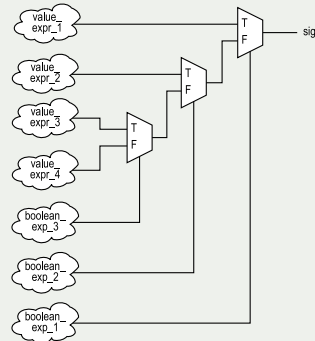
- A implementação do *IF* é a mesma que atribuição CONDICIONAL se consistir em:
 - Um sinal de saída
 - Uma atribuição de sinal sequencial em cada ramo



Implementação Conceitual

- A implementação do *IF* é a mesma que atribuição CONDICIONAL se consistir em:
 - Um sinal de saída
 - Uma atribuição de sinal sequencial em cada ramo
- Exemplo:

```
1      if boolean_expr_1 then
2          sig <= value_expr_1;
3      elsif boolean_expr_2 then
4          sig <= value_expr_2;
5      elsif boolean_expr_3 then
6          sig <= value_expr_3;
7      else
8          sig <= value_expr_4;
9      end if;
```



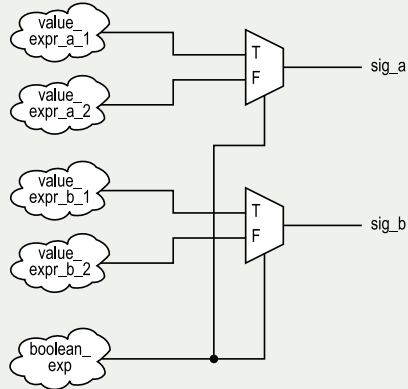
Exemplo: Dois sinais de saída a mesma expressão booleana.

```
1  if boolean_expr then
2      sig_a <= value_expr_a_1;
3      sig_b <= value_expr_b_1
4  else
5      sig_a <= value_expr_a_2;
6      sig_b <= value_expr_b_2 ;
7  end if;
```



Exemplo: Dois sinais de saída a mesma expressão booleana.

```
1  if boolean_expr then
2    sig_a <= value_expr_a_1;
3    sig_b <= value_expr_b_1
4  else
5    sig_a <= value_expr_a_2;
6    sig_b <= value_expr_b_2 ;
7  end if;
```



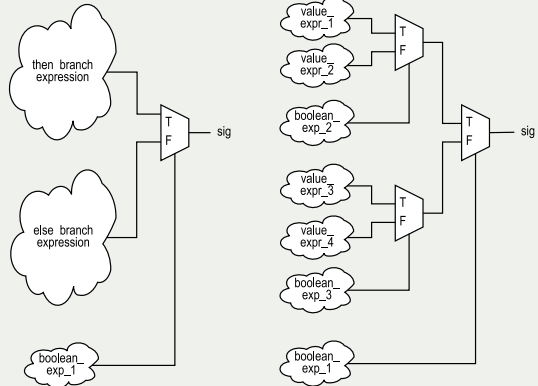
Exemplo: Dois níveis de *IFs* aninhados.

```
1  if boolean_expr_1 then
2      if boolean_expr_2 then
3          signal_a <= value_expr_1;
4      else
5          signal_a <= value_expr_2;
6      end if;
7  else
8      if boolean_expr_3 then
9          signal_a <= value_expr_3;
10     else
11         signal_a <= value_expr_4;
12     end if;
13 end if;
```



Exemplo: Dois níveis de IFs aninhados.

```
1  if boolean_expr_1 then
2    if boolean_expr_2 then
3      signal_a <= value_expr_1;
4    else
5      signal_a <= value_expr_2;
6    end if;
7  else
8    if boolean_expr_3 then
9      signal_a <= value_expr_3;
10   else
11     signal_a <= value_expr_4;
12   end if;
13 end if;
```



Declaração CASE

■ Sintaxe:

```
1      case case_expression is
2          when choice_1 =>
3              sequential statements;
4          when choice_2 =>
5              sequential statements;
6
7          . . .
8
9          when choice_n =>
10             sequential statements;
11      end case;
```



■ Sintaxe:

```
1      case case_expression is
2          when choice_1 =>
3              sequential statements;
4          when choice_2 =>
5              sequential statements;
6
7          . . .
8
9          when choice_n =>
10             sequential statements;
11      end case;
```

■ Exemplo (Mux 4x1):

```
1      architecture case_arch of mux4 is
2      begin
3          process(a,b,c,d,s)
4          begin
5              case s is
6                  when "00" =>
7                      x <= a;
8                  when "01" =>
9                      x <= b;
10                 when "10" =>
11                     x <= c;
12                 when others =>
13                     x <= d;
14             end case;
15         end process;
16     end case_arch;
```



- As duas estrutura são iguais se houver apenas um sinal de saída no *CASE*.
- Declaração *CASE* é mais flexível.
- Declarações sequenciais podem ser utilizadas nas opções do *CASE*.



CASE vs. atribuição de SELEÇÃO (*with select when*)

■ Atribuição de SELEÇÃO:

```
1  with sel_exp select
2      sig <= value_expr_1 when choice_1,
3          value_expr_2 when choice_2,
4          value_expr_3 when choice_3,
5
6          . . .
7
8          value_expr_n when choice_n;
9
```



CASE vs. atribuição de SELEÇÃO (with select when)

■ Atribuição de SELEÇÃO:

```
1  with sel_exp select
2      sig <= value_expr_1 when choice_1,
3          value_expr_2 when choice_2,
4          value_expr_3 when choice_3,
5
6      . . .
7
8      value_expr_n when choice_n;
9
```

■ Pode ser escrita como:

```
1  process ( ... )
2  begin
3      case sel_exp is
4          when choice_1 =>
5              sig <= value_expr_1;
6          when choice_2 =>
7              sig <= value_expr_2;
8          when choice_3 =>
9              sig <= value_expr_3;
10
11         . . .
12
13         when choice_n =>
14             sig <= value_expr_n;
15     end case;
16 end process;
17
```



Atribuição de sinal incompletos

- De acordo com a definição VHDL:
 - Os sinais não precisam ser atribuídos em todos os ramos;
 - Quando um sinal não é atribuído por omissão, ele mantém o “valor anterior” (implicando em “memória”).
- Exemplo:



Atribuição de sinal incompletos

- De acordo com a definição VHDL:
 - Os sinais não precisam ser atribuídos em todos os ramos;
 - Quando um sinal não é atribuído por omissão, ele mantém o “valor anterior” (implicando em “memória”).
- Exemplo:

```
1  process(a)
2  begin
3      case a is
4          when "100"|"101"|"110"|"111" =>
5              high <= '1';
6          when "010"|"011" =>
7              middle <= '1';
8          when others =>
9              low <= '1';
10     end case;
11 end process;
```



■ Correção 1:

```
1  process(a)
2  begin
3      case a is
4          when "100"|"101"|"110"|"111" =>
5              high <= '1';
6              middle <= '0';
7              low <= '0';
8          when "010"|"011" =>
9              high <= '0';
10             middle <= '1';
11             low <= '0';
12         when others =>
13             high <= '0';
14             middle <= '0';
15             low <= '1';
16     end case;
17 end process;
```



■ Correção 1:

```
1 process(a)
2 begin
3   case a is
4     when "100"|"101"|"110"|"111" =>
5       high <= '1';
6       middle <= '0';
7       low <= '0';
8     when "010"|"011" =>
9       high <= '0';
10      middle <= '1';
11      low <= '0';
12     when others =>
13       high <= '0';
14       middle <= '0';
15       low <= '1';
16   end case;
17 end process;
```

■ Correção 2:

```
1 process(a)
2 begin
3   high <= '1';
4   middle <= '0';
5   low <= '0';
6   case a is
7     when "100"|"101"|"110"|"111" =>
8       high <= '1';
9     when "010"|"011" =>
10      middle <= '1';
11     when others =>
12      low <= '1';
13   end case;
14 end process;
```

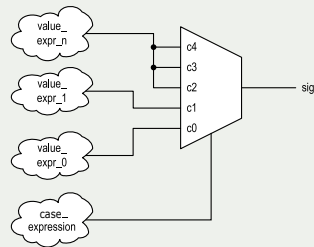


- A implementação do *CASE* é a mesma que atribuição de *SELEÇÃO* se consistir em:
 - Um sinal de saída
 - Uma atribuição de sinal sequencial em cada ramo



- A implementação do *CASE* é a mesma que atribuição de *SELEÇÃO* se consistir em:
 - Um sinal de saída
 - Uma atribuição de sinal sequencial em cada ramo
 - Exemplo:

```
1 case case_exp is
2   when c0 =>
3     sig <= value_expr_0 ;
4   when c1 =>
5     sig <= value_expr_1 ;
6   when others =>
7     sig <= value_expr_n ;
8 end case;
```



- Várias instruções sequenciais podem ser construídas recursivamente.

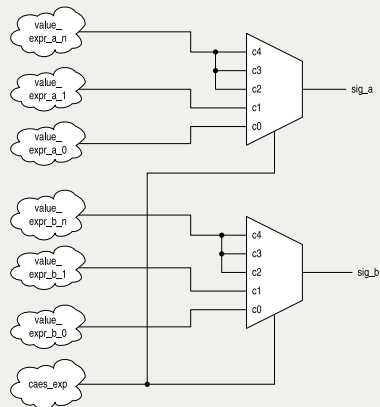
Exemplo: Dois sinais de saída a mesma expressão booleana.

```
1 case case_exp is
2   when c0 =>
3     sig_a <= value_expr_a_0;
4     sig_b <= value_expr_b_0;
5   when c1 =>
6     sig_a <= value_expr_a_1;
7     sig_b <= value_expr_b_1;
8   when others =>
9     sig_a <= value_expr_a_n;
10    sig_b <= value_expr_b_n;
11 end case;
```



Exemplo: Dois sinais de saída a mesma expressão booleana.

```
1 case case_exp is
2   when c0 =>
3     sig_a <= value_expr_a_0;
4     sig_b <= value_expr_b_0;
5   when c1 =>
6     sig_a <= value_expr_a_1;
7     sig_b <= value_expr_b_1;
8   when others =>
9     sig_a <= value_expr_a_n;
10    sig_b <= value_expr_b_n;
11 end case;
```



Declaração *FOR LOOP* simples

- VHDL fornece uma variedade de construções de loop
- Apenas uma forma restrita de loop pode ser sintetizada
- Sintaxe:

```
1 for index in loop_range loop  
2     sequential statements;  
3 end loop;
```

- loop_range deve ser estático.
- index assume o valor de loop_range da esquerda para a direita.



Sintaxe e Exemplo

- VHDL fornece uma variedade de construções de loop
- Apenas uma forma restrita de loop pode ser sintetizada
- Sintaxe:

```
1 for index in loop_range loop
2     sequential statements;
3 end loop;
```

- loop_range deve ser estático.
- index assume o valor de loop_range da esquerda para a direita.

■ Exemplo XOR bit a bit:

```
1 architecture demo_arch of bit_xor is
2     constant WIDTH: integer := 4;
3 begin
4     process(a,b)
5     begin
6         for i in (WIDTH-1) downto 0 loop
7             y(i) <= a(i) xor b(i);
8         end loop;
9     end process;
10 end demo_arch;
```



Sintaxe e Exemplo

- VHDL fornece uma variedade de construções de loop
- Apenas uma forma restrita de loop pode ser sintetizada
- Sintaxe:

```
1 for index in loop_range loop
2     sequential statements;
3 end loop;
```

- loop_range deve ser estático.
- index assume o valor de loop_range da esquerda para a direita.

■ Exemplo XOR bit a bit:

```
1 architecture demo_arch of bit_xor is
2     constant WIDTH: integer := 4;
3 begin
4     process(a,b)
5     begin
6         for i in (WIDTH-1) downto 0 loop
7             y(i) <= a(i) xor b(i);
8         end loop;
9     end process;
10 end demo_arch;
```

- Mesmo que $y \leq a \text{ xor } b$;



- “Desenrole” o *LOOP*.
- O *FOR LOOP* deve ser tratado como “abreviação” para declarações repetitivas.
- Exemplos (*XOR* bit a bit):

```
1 y(3) <= a(3) xor b(3);  
2 y(2) <= a(2) xor b(2);  
3 y(1) <= a(1) xor b(1);  
4 y(0) <= a(0) xor b(0);
```



Conclusão

- Declarações concorrentes:
 - Modelado conforme hardware;
 - Tem mapeamento claro e direto para estruturas físicas.



Síntese de declarações sequenciais

- Declarações concorrentes:
 - Modelado conforme hardware;
 - Tem mapeamento claro e direto para estruturas físicas.
- Declarações sequenciais:
 - Destina-se a descrever “comportamento”;
 - Flexível e versátil;
 - Pode ser difícil de ser implementado em hardware;
 - Pode-se facilmente perder o controle.



Síntese de declarações sequenciais

- Declarações concorrentes:
 - Modelado conforme hardware;
 - Tem mapeamento claro e direto para estruturas físicas.
- Declarações sequenciais:
 - Destina-se a descrever “comportamento”;
 - Flexível e versátil;
 - Pode ser difícil de ser implementado em hardware;
 - Pode-se facilmente perder o controle.

- Pense em hardware.
- Projetar hardware não é converter um programa C em VHDL.



■ Para instruções sequenciais:

- As variáveis devem ser usadas com cuidado. Geralmente, um sinal é preferido. Uma declaração como $n := n + 1$ pode causar grande confusão para síntese.
- Exceto para o valor padrão, evite sobrescrever um sinal várias vezes em um processo.
- Pense nas instruções *IF* e *CASE* como estruturas de roteamento em vez de construções de controle sequenciais.



- Para instruções sequenciais (cont.):
 - Um *IF* infere uma estrutura de roteamento de prioridade e um grande número de ramificações leva a uma longa cascata.
 - Um *CASE* infere uma estrutura de multiplexação e um grande número de opções leva a um multiplexador com muitas entradas.
 - Pense em um *LOOP* como um mecanismo para replicar uma estrutura.
 - Evite o uso “inovador” de construções do VHDL. Devemos ser inovadores quanto à arquitetura de hardware, mas não quanto à descrição da arquitetura. O software de síntese pode não ser capaz de interpretar a intenção do código.



■ Para **circuitos combinacionais**:

- Inclua todos os sinais de entrada na lista de sensibilidade para evitar comportamento inesperado.
- Para o *IF*, sempre inclua o *ELSE* para evitar inferência de memória;
- Um sinal de saída deve receber um valor para cada seção do *IF*. Uma boa prática é atribuir um valor padrão a cada sinal no início do processo.



FIM!